Thema: KI basiertes Programmieren: Entwicklung einer React-Web App
Name des Moduls: [SoSe26] Basissysteme der Informationsverarbeitung 2
Art der Leistung: Studienleistung #1
Team: JEBA-Gruppe: Benedikt Benz [bbenz], Joris Kasten [jkasten], Eric Steffens [esteffe2], Alic Malorodow [amalorod]
Abgabedatum: 05.05.2026

E-Navigator – KI-gestützte interaktive Web-App zur Anzeige von E-Ladestationen

Link zum GitHub-Repository: https://github.com/amalorod/E-Navigator
Clone URL: https://github.com/amalorod/E-Navigator.git

Inhalt

1. Beschreibung der Anwendung 2
Zentrale Funktionen 2
2. Verwendete APIs 3
2.1 Geocoding mit Nominatim 3
2.2 Ladesäulendaten der Bundesnetzagentur 3
3. Eingesetzte Kartenbibliothek 4
4. Technische Umsetzung 5
4.1 Navigation 5
4.2 Datenabruf 5
4.3 Hilfsfunktionen für Ladesäulen 6
4.4 Karteninteraktion 6
5. Eingesetzte KI-Tools 6
6. Dokumentierte Prompts 7
7. Reflexion zur KI-Nutzung 9
8. Fazit 10

1. Beschreibung der Anwendung

Der **E-Navigator** ist eine Webanwendung, mit der Nutzerinnen und Nutzer öffentliche E-Ladestationen in Deutschland suchen und auf einer interaktiven Karte anzeigen können. Die Anwendung wurde als **Vite React TypeScript Projekt** umgesetzt und besteht aus mehreren Views: einer Startseite, einer Suchseite und einer Kartenansicht. Ziel der Anwendung ist es, standortbezogene Daten aus externen Schnittstellen dynamisch zu laden, sinnvoll aufzubereiten und nutzerfreundlich auf einer Karte darzustellen. Die Anwendung verbindet dabei eine klassische Adresssuche mit einer interaktiven Kartenansicht. Nutzerinnen und Nutzer können entweder eine Adresse eingeben oder direkt auf die Karte klicken. Anschließend werden passende Ladesäulen in der Umgebung geladen und als Marker dargestellt.
Die Anwendung erfüllt damit mehrere zentrale Anforderungen: Sie nutzt externe REST-APIs, verarbeitet die geladenen Daten dynamisch, stellt standortbezogene Informationen auf einer Karte dar und bietet eine Navigation zwischen verschiedenen Ansichten.

Zentrale Funktionen

Die Anwendung ist in drei Hauptbereiche gegliedert:

1. **Home View**
   Die Startseite gibt eine kurze Einführung in die App. Sie erklärt den Zweck der Anwendung und zeigt den Nutzen: Das schnelle Finden von öffentlichen E-Ladestationen in der Nähe eines bestimmten Ortes.

2. **Search View**
   In der Suchansicht kann eine Adresse über ein Eingabefeld gesucht werden. Während der Eingabe werden passende Adressvorschläge über die Geocoding-API geladen und in einem Dropdown angezeigt. Nach Auswahl einer Adresse werden Ladestationen in der Umgebung geladen. Zusätzlich kann eine Ergebnisliste mit Informationen zu den gefundenen Ladesäulen angezeigt werden.

3. **Map View**
   Die Kartenansicht zeigt die gefundenen Ladesäulen auf einer interaktiven Leaflet-Karte. Die Marker werden geclustert, damit auch größere Mengen an Punkten übersichtlich dargestellt werden können. Beim Klick auf einen Marker erscheinen Detailinformationen in einem Popup. Zusätzlich wurde die Karteninteraktion erweitert: Klickt man direkt auf eine Stelle in der Karte, wird dort ein Marker gesetzt und es werden Ladesäulen im Umkreis von 10 km geladen. Bei einem neuen Klick verschwinden die bisherigen Ergebnisse und der Umkreis wird neu berechnet.

Durch diese Struktur entsteht eine klare und benutzerfreundliche Anwendung: Nutzerinnen und Nutzer können entweder über die Suche starten oder direkt über die Karte interagieren.

2. Verwendete APIs

Die Anwendung verwendet externe Schnittstellen, um sowohl Adressen als auch Ladesäulen dynamisch zu laden.

2.1 Geocoding mit Nominatim

Für die Adresssuche wird die **Nominatim Search API** verwendet. Nominatim erlaubt die Suche nach Orten oder Adressen über eine textuelle Eingabe. Laut Dokumentation besitzt die Search API das Format https://nominatim.openstreetmap.org/search?<params> und unterstützt unter anderem freie Suchanfragen über den Parameter q. Außerdem kann über den Parameter format ein Ausgabeformat wie jsonv2 gewählt werden (https://nominatim.org/release-docs/latest/api/Search/).

Im Projekt wird Nominatim genutzt, um aus einer eingegebenen Adresse geografische Koordinaten zu erhalten. Die Antwort enthält unter anderem Werte wie lat, lon und display_name. Diese Daten werden in der Anwendung in ein eigenes Format umgewandelt:

```
1 {
2   lat: Number(item.lat),
3   lon: Number(item.lon),
4   displayName: item.display_name
5 }
```

Damit kann die App die gewählte Adresse auf der Karte markieren und anschließend Ladesäulen in der Nähe laden.

2.2 Ladesäulendaten der Bundesnetzagentur

Die Anwendung basiert auf öffentlichen Daten zu Ladeeinrichtungen in Deutschland. Die Bundesnetzagentur veröffentlicht Daten zur öffentlich zugänglichen Ladeinfrastruktur im Rahmen der Ladesäulenverordnung. Die Ladesäulenkarte der Bundesnetzagentur stellt Ladeeinrichtungen dar, deren Anzeigeverfahren vollständig abgeschlossen wurde. Zu den per Klick abrufbaren Informationen gehören laut Bundesnetzagentur unter anderem Betreiber, Adresse, Public Key und technische Ausstattung:
https://www.bundesnetzagentur.de/DE/Fachthemen/ElektrizitaetundGas/E-Mobilitaet/Ladesaeulenkarte/start.html.

Ursprünglich war für das Projekt der Endpunkt https://ladestationen.api.bund.dev/query vorgesehen. Die OpenAPI-Dokumentation beschreibt dort einen /query-Endpunkt für alle Ladesäulen. Relevante Parameter sind unter anderem geometry, geometryType, f, outFields, returnGeometry, spatialRel, inSR und outSR. Außerdem werden Ausgabeformate wie json, geojson, pbf und html genannt.

Während der Entwicklung trat jedoch ein 404-Fehler beim Abruf über diesen Endpunkt auf. Deshalb wurde im aktuellen Arbeitsstand ein ArcGIS FeatureServer als technische Alternative verwendet. Ein gefundener FeatureServer beschreibt den Dienst **Ladesaeulen_in_Deutschland** als Datenbestand gemeldeter Ladesäulen in Deutschland, bereitgestellt durch die Bundesnetzagentur. Der Dienst enthält einen Layer Ladesäulen_072025 (0) und unterstützt Query-Operationen.

Im Code wird für den Abruf der Ladesäulen eine Bounding Box erzeugt. Dadurch wird nicht der gesamte Datenbestand geladen, sondern nur ein geografischer Ausschnitt. Dies verbessert die Performance und macht die Darstellung auf der Karte praktikabler.

Beispielhaft wird eine Bounding Box um einen Punkt erzeugt:

```
const bbox = getBboxAroundPoint(lat, lon, 10);
const collection = await
```

Anschließend werden die Ergebnisse zusätzlich nach echter Entfernung gefiltert. Dadurch wird aus der rechteckigen API-Abfrage ein nutzerfreundlicher 10-km-Umkreis.

3. Eingesetzte Kartenbibliothek

Für die Kartenintegration wurde **Leaflet** eingesetzt. Leaflet ist eine JavaScript-Bibliothek zur Darstellung interaktiver Karten. In der Anwendung wird Leaflet verwendet, um die Karte zu initialisieren, OpenStreetMap-Kacheln einzubinden, Marker zu setzen und Popups anzuzeigen.

Zusätzlich wurde das Plugin **Leaflet.markercluster** verwendet. Das GitHub-Projekt beschreibt es als Plugin, das animiertes Marker-Clustering für Leaflet bereitstellt. Laut npm-Dokumentation können Marker in einer MarkerClusterGroup gesammelt und anschließend der Karte hinzugefügt werden:

https://github.com/Leaflet/Leaflet.markercluster
https://www.npmjs.com/package/leaflet.markercluster

Der Einsatz von Clustering ist für diese Anwendung besonders sinnvoll, weil in Ballungsräumen sehr viele Ladesäulen auf engem Raum liegen können. Ohne Clustering würden sich Marker überlagern und die Karte wäre schnell unübersichtlich. Durch Clustering werden nahe beieinanderliegende Marker zunächst gruppiert. Beim Zoomen oder Anklicken werden die einzelnen Stationen differenzierter sichtbar.
In der Anwendung werden die geladenen Ladesäulen daher nicht direkt einzeln der Karte hinzugefügt, sondern über eine Cluster-Layer-Struktur verwaltet. Bei jeder neuen Suche oder jedem neuen Kartenklick werden die alten Marker entfernt und durch neue Marker ersetzt.

4. Technische Umsetzung

Die Anwendung wurde als React-Projekt mit TypeScript umgesetzt. Die Struktur ist modular aufgebaut:

```
1  src/
2    components/
3      NavBar.tsx
4    hooks/
5      useHashView.ts
6    pages/
7      HomePage.tsx
8      SearchPage.tsx
9      MapPage.tsx
10   services/
11     api.ts
12   utils/
13     stations.ts
14   App.tsx
15   App.css
16   index.css
17   types.ts
```

4.1 Navigation

Die Navigation erfolgt über einen eigenen Hook useHashView. Dieser liest den aktuellen Hash aus der URL aus und setzt abhängig davon die aktive View. Dadurch kann ohne zusätzliches Routing-Paket zwischen home, search und map gewechselt werden.

Die Komponente NavBar.tsx stellt die Navigationsbuttons dar. Je nach aktivem View wird der entsprechende Button hervorgehoben.

4.2 Datenabruf

Der API-Zugriff ist in services/api.ts gekapselt. Dort befinden sich Funktionen für:

- Adresssuche über Nominatim
- Abruf von Ladesäulen über eine geografische Bounding Box
- Umwandlung der API-Antwort in ein einheitliches internes Format

Die Trennung in eine eigene Service-Datei macht den Code übersichtlicher und erleichtert spätere Änderungen an der API.

4.3 Hilfsfunktionen für Ladesäulen

In utils/stations.ts wurden zentrale Hilfsfunktionen ausgelagert. Dazu gehören Funktionen zum Ermitteln einer stabilen ID, zum Auslesen des Betreibers, zum Formatieren der Adresse und zur Berechnung von Entfernungen.
Besonders wichtig war die robuste Behandlung unterschiedlicher Feldnamen. Da externe API-Daten nicht immer exakt die erwarteten Property-Namen liefern, wurden Funktionen entwickelt, die verschiedene Schreibweisen berücksichtigen. Dadurch kann die Anwendung beispielsweise Felder wie strasse, Straße, STRASSE oder ähnliche Varianten besser verarbeiten.

4.4 Karteninteraktion

Die Kartenansicht wurde so erweitert, dass nicht nur Ergebnisse aus der Suchseite angezeigt werden können. Nutzerinnen und Nutzer können auch direkt auf die Karte klicken. Beim Klick wird:

1. der bisherige Suchmarker entfernt,
2. ein neuer Marker am Klickpunkt gesetzt,
3. die bisherigen Ladesäulenmarker gelöscht,
4. eine neue Bounding Box für 10 km Umkreis berechnet,
5. die API neu abgefragt,
6. die Ergebnisse nach echter Entfernung gefiltert,
7. die neuen Marker auf der Karte dargestellt.

Dadurch entsteht eine dynamische, intuitive Karteninteraktion.

5. Eingesetzte KI-Tools

Die Entwicklung erfolgte KI-gestützt. Verwendet wurden mehrere KI-Werkzeuge innerhalb und außerhalb von Visual Studio Code. Insbesondere kamen GitHub Copilot und weitere KI-Assistenzsysteme zum Einsatz.

Die KI wurde für verschiedene Aufgaben genutzt:

- Erstellung der Grundstruktur des Vite React TypeScript Projekts
- Entwicklung der drei Views
- Integration der API-Aufrufe
- Einbindung von Leaflet und Leaflet.markercluster
- Verbesserung von Styling und Responsivität
- Fehleranalyse und Debugging
- Ergänzung von Fehlerbehandlung
- Erweiterung der Karteninteraktion
- Refactoring einzelner Codeabschnitte

Die KI wurde nicht nur zur initialen Codegenerierung genutzt, sondern auch zur Weiterentwicklung bestehender Komponenten. Besonders hilfreich war dies bei kleineren Änderungen und Fehlersituationen, zum Beispiel beim Problem mit nicht

funktionierenden Kartenmarkierungen. Dort wurde erkannt, dass vorgeschlagene Platzhalterfunktionen mit throw new Error ('Function not implemented.') keine echte Funktionalität besitzen. Die Lösung bestand darin, die echten React-State-Setter aus App.tsx an die MapPage zu übergeben, zum Beispiel onCenterChange={setCenter} und onStationsLoaded={setStations}.

6. Dokumentierte Prompts

Im Folgenden sind die wesentlichen Prompts einheitlich nummeriert zusammengefasst.

Prompt 1: Grundidee und Projektanforderungen
Aktuell liegt im Repository E-Navigator ein neu aufgesetztes Vite React Projekt mit TypeScript. Ich möchte ein React Projekt bauen, das mindestens ein externes REST API einbindet, Daten dynamisch lädt, eine Such-, Filter- oder Auswahlfunktion enthält und eine interaktive Karte integriert. Die Anwendung soll mindestens drei Views enthalten: Home View, Search View und Map View. Meine Idee ist eine App, mit der man E-Ladestationen dynamisch auf einer Karte abrufen kann. Als Datenquelle dient https://ladestationen.api.bund.dev/. Die dynamischen Ladesäulendaten der Bundesnetzagentur sollen per fetch geladen werden. Für die Kartendarstellung soll Leaflet mit Leaflet.markercluster verwendet werden.

Prompt 2: Erstellung der Grundstruktur
Erstelle ein Vite React TypeScript Projekt mit drei Views: Home, Suche und Karte. Baue eine Navigation zwischen diesen Views ein und strukturiere die Anwendung so, dass sie benutzerfreundlich bedienbar ist.

Prompt 3: API-Integration
Integriere die Bundesnetzagentur-Ladestations-API über fetch und zeige Ergebnisse als Liste an. Die Suchseite soll auf derselben Datengrundlage basieren wie die Kartenseite. Über ein Input-Feld sollen Adressen eingegeben werden können, und anschließend sollen passende Ladesäulen mit Informationen angezeigt werden
.

Prompt 4: Kartenintegration
Baue eine Leaflet-Karte mit Leaflet.markercluster und verknüpfe die Listenauswahl mit Marker und Centering. Bei Auswahl eines Elements aus der Liste soll die Karte passend reagieren, zum Beispiel zu einem Ort springen, einen Marker hervorheben und Detailinformationen anzeigen.

Prompt 5: Verbesserung der Anwendung
Mache die App besser.
Dieser Prompt führte zu Verbesserungen in Struktur, Benutzerführung, Styling und Wiederverwendbarkeit einzelner Funktionen.

Prompt 6: Fehlerbehandlung
Verbessere das Fehlerhandling.

Daraufhin wurden Ladezustände, Fehlermeldungen und robustere Abfragen ergänzt. Außerdem wurden API-Antworten stärker geprüft, damit leere oder unerwartete Antworten nicht zu Abstürzen führen.

Prompt 7: Fehleranalyse bei fehlenden API-Daten
Schaue dir den aktuellen Commit nochmal an und arbeite im aktuellen Commit weiter. Aktuell sehe ich keine Daten, die aus der API der Bundesnetzagentur geladen werden. Ich will, dass man im View „Suche“ eine Adresse eintippen kann. Diese erscheint im Dropdown-Menü. Nach Klick auf diese Adresse gelangt man auf die Karten-View und sieht dort die zuvor angeklickte Adresse inklusive Standorte für die Ladesäulen.
Dieser Prompt führte zu einer genaueren Analyse des Datenflusses zwischen Suche, App-State und Karte. Außerdem wurde die Navigation verbessert, damit die Kartenansicht erst nach dem Laden relevanter Daten geöffnet wird.

Prompt 8: Verbesserung der Benutzeroberfläche
Verbessere den Kontrast der Schrift und der Buttons. Füge die Navigationsleiste unterhalb der Überschrift ein und positioniere alles wie bei einer professionellen Web-App.

Daraufhin wurden CSS-Struktur, Kontraste, Layout, Panels, Buttons, Eingabefelder und responsive Eigenschaften überarbeitet.

Prompt 9: Erweiterung der Suchergebnisse
Ich will auf meiner Seite in der Suchfunktion nach eingegebenem Standort eine Liste der nächsten Ladesäulen angezeigt bekommen mit Entfernung, Adresse und ein paar wichtigen Informationen zur Ladesäule.

Die KI analysierte den bestehenden Code und ergänzte eine Entfernungsermittlung. Die Ladesäulen wurden nach Entfernung sortiert und mit zusätzlichen Informationen angezeigt.

Prompt 10: Standortfunktion
Ich will, dass man seinen Standort bestimmen lassen kann mit einem Button. Daraufhin wurde ein Button „Mein Standort“ für die Suchseite vorgesehen beziehungsweise umgesetzt, mit dem der aktuelle Standort als Ausgangspunkt für die Suche genutzt werden kann.

Prompt 11: Kartenklick als Suchauslöser
Ich will, dass mit dem Klick auf die Karte, also ohne vorher Adressen eingegeben zu haben, alle Ladesäulen im Umkreis von 10 km erscheinen. Man klickt auf die Karte, es erscheint ein Marker und drumherum alle Adressen. Klickt man auf eine Stelle daneben, dann verschwinden die vorherigen Adressen und der Umkreis wird neu geladen.

Dieser Prompt führte zur Erweiterung der Kartenansicht um eine direkte Karteninteraktion. Die Karte kann nun selbst eine Suche auslösen, ohne dass vorher eine Adresse eingegeben werden muss.

Prompt 12: Fehlerbehebung bei Kartenmarkierung
Copilot hat einen Error markiert und Platzhalterfunktionen mit throw new Error ('Function not implemented.') vorgeschlagen. Die Markierung auf der Karte funktioniert nicht.

Dieser Prompt führte zur Korrektur der Props in App.tsx. Statt Platzhalterfunktionen wurden echte React-State-Setter an die Kartenkomponente übergeben.

7. Reflexion zur KI-Nutzung

Der Einsatz mehrerer KI-Werkzeuge innerhalb und außerhalb von Visual Studio Code war für die Entwicklung sehr hilfreich. Besonders bei der Erstellung größerer Codepakete konnte die KI schnell Grundstrukturen erzeugen, etwa für die Views, die Navigation, die API-Services und die Leaflet-Karte. Dadurch war es möglich, in kurzer Zeit eine lauffähige Anwendung aufzubauen.

Auch bei kleineren Änderungen war die KI nützlich. Ein konkretes Beispiel war die Fehlerbehebung bei der Kartenmarkierung: Copilot hatte automatisch Platzhalterfunktionen vorgeschlagen, die jeweils throw new Error('Function not implemented.') enthielten. Diese Funktionen sahen formal passend aus, führten aber dazu, dass beim Klick auf die Karte ein Fehler ausgelöst wurde. Mithilfe der KI konnte nachvollzogen werden, dass diese Platzhalter durch echte State-Setter ersetzt werden mussten. Danach wurden beispielsweise setCenter, setSelectedAddress, setStations und setSelected korrekt an die MapPage übergeben.

Ein weiteres Beispiel war die Verarbeitung uneinheitlicher API-Daten. Da die Feldnamen externer Datenquellen nicht immer exakt den erwarteten Namen entsprechen, unterstützte die KI dabei, Hilfsfunktionen zu entwickeln, die unterschiedliche Schreibweisen berücksichtigen. Dadurch wurde die Anwendung robuster gegenüber Änderungen oder Abweichungen in den API-Daten.

Positiv war außerdem, dass die KI nicht nur neuen Code erzeugt hat, sondern auch bestehende Probleme erklären konnte. Bei Fehlern wie fehlenden API-Daten, leeren Ergebnislisten oder nicht sichtbaren Markern half die KI dabei, systematisch zwischen API-Problem, Datenformatproblem und Darstellungsproblem zu unterscheiden.

Gleichzeitig zeigte die Entwicklung auch Grenzen der KI-Nutzung. Nicht jeder KI-Vorschlag war direkt verwendbar. Manche Vorschläge mussten geprüft und angepasst werden, insbesondere wenn automatisch generierter Code nur Platzhalter enthielt oder nicht exakt zur bestehenden Projektstruktur passte. Dadurch wurde deutlich, dass KI zwar stark bei Umsetzung und Fehlersuche unterstützt, die Ergebnisse aber trotzdem kontrolliert und verstanden werden müssen.

Insgesamt hat die KI den Entwicklungsprozess stark beschleunigt. Sie half bei der Planung, Codegenerierung, Fehleranalyse, Umstrukturierung und Dokumentation. Gleichzeitig war es notwendig, die generierten Lösungen zu testen, Fehlermeldungen zu interpretieren und einzelne Codeabschnitte gezielt anzupassen. Die Arbeit mit KI

war daher besonders effektiv, wenn die Prompts möglichst konkret formuliert wurden und die erzeugten Ergebnisse anschließend kritisch geprüft wurden.

8. Fazit

Der E-Navigator zeigt, wie sich mit React, TypeScript, externen APIs und einer Kartenbibliothek eine praxisnahe Webanwendung entwickeln lässt. Die Anwendung lädt dynamisch Daten, verarbeitet geografische Informationen und stellt diese nutzerfreundlich auf einer interaktiven Karte dar.

Die KI-gestützte Entwicklung ermöglichte eine schnelle Umsetzung der Grundfunktionen und half bei der Erweiterung der App um zusätzliche Interaktionen wie Adresssuche, Ergebnisliste, Marker-Clustering und Kartenklick-Suche. Besonders wertvoll war die KI bei Debugging-Aufgaben und beim Refactoring einzelner Codeabschnitte.

Trotzdem zeigte sich, dass KI-generierter Code nicht ungeprüft übernommen werden sollte. Gerade bei API-Integrationen, externen Datenformaten und React-State-Übergaben ist eine manuelle Kontrolle wichtig. Die Kombination aus KI-Unterstützung und eigenem Testen führte schließlich zu einer stabileren und besser strukturierten Anwendung.